COMPARISON OF LOSSLESS DATA COMPRESSION ALGORITHMS FOR TEXT DATA

Nayan Patel (UMID: 27900573)
nayanpat@umich.edu
CIS 505 Project (Item 1), Fall 2021 Term
Faculty: Prof. Thomas Steiner

Table of Contents

1. Problem Statement 2
2. Solution Methods 2
   2.1 Huffman Algorithm 2
   2.2 Shannon-Fano Algorithm 4
3. Analysis of Used Algorithms 6
4. Result Prediction and Preliminary Recommendation 7
5. Performance Metrics for Algorithm Evaluation 7
6. Design Consideration 8
7. Test Plan 8
   7.1 Purpose 8
   7.2 Testing Types 8
      7.2.1 Unit Testing 8
      7.2.2 Functional Testing 9
8. Source Code of Programs 10
9. Build Results and Output of Programs 20
10. Result Analysis 27
11. Conclusion 28

1. Problem Statement

The storage devices continue to fill up due to the increased storage demand as the generation of number of data files are growing exponentially. Given a data file, it would therefore be desirable to find a way to store the file as efficiently as possible. Data compression is a way to achieve such a goal and the problem of data compression can be solved by various encoding algorithms.

Each encoding algorithm has its own advantages and disadvantages. The selection of algorithm to use is based on different criteria e.g., optimality, coding efficiency, compression ratio, time complexity etc.

In this project, problem of data compression is solved using two encoding algorithms and their performance are compared side-by-side for said criteria and evaluated based on actual implementation.

2. Solution Methods

One of the most popular algorithms that is used for encoding data files is Huffman Coding which is a greedy algorithm generating variable-length bit sequence for each symbol in the data file. It provides prefix codes and hence ensures lossless data compression and prevents ambiguity.

Shannon-Fano Coding is another similar algorithm that is also a lossless data compression method, produces variable-length bit sequences for symbols in the file.

In this experiment, **Huffman and Shannon Fano Coding** are used to encode the same data from text files and their performance are compared side-by-side. Four different text files of different sizes (a few kilobytes to several megabytes) are used as inputs to compare the performance of the algorithms.

2.1 Huffman Algorithm

Following pseudo-code represents the Huffman Code algorithm:

```
// Structure to store a node information
struct node
{
        char symbol
        float frequency

        node left
        node right
}

// Structure to compare two node frequency values
struct compare
{
        If (node left > node right)
                return True
}

//Procedure to build Huffman Tree
buildHuffmanTree (sym[], freq[], n)
{
        Priority_queue PQ
        node left, right, newNode
```

```
        for (i = 1 to n)
                Add new node(ith sym, ith freq) -> PQ

        while(size of PQ is not equal to 1)
        {
                left = top node of PQ
                remove top node

                right = top node of PQ
                remove top node

                newNode  = left node Frequency + right node Frequency

                Add newNode to PQ
        }
}
```

// Main() Procedure

```
main(argc, argv)
{
        char sym[]
        float freq[]
        int occurance[]

        // File text to encode
        filename = read from the command line argument

        // read each character and its occurrence from the file
        while(till character in filename)
        {
                ith sym = character
                ith occurance = occurrence of character
        }

        n = no. of characters

        // calculate total occurrence
        for(i = 1 to n)
                total occurrence =  total occurrence + occurrence of ith character

        // calculate frequency of each character
        for(i = 1 to n)
                ith freq = occurrence of ith character / total occurrence

        // calculate entropy of each character
        for(i = 1 to n)
                entropy = entropy + (-ith freq * log2 of ith freq)

        // calculate minimum bit length required to store each symbol based on fixed length coding
        for(i = 1 to n)
        {
                if(2^i >= n)
                    fixed length code = i
        }
```

```
        // Call Huffman code procedure
        buildHuffmanTree(sym, freq, n)

        // Calculate following statistics
        Total bit size of original data before encoding = total occurrences * 8 bits/ASCII character
        Total bit size of encoded data = total occurrences each character * encoded bit size of that character
        Efficiency of fixed length code = entropy / fixed length code
        Efficiency of encoded characters = entropy / average bit size of encoded characters
        Compression ratio  = Total bit size of original data before encoding / Total bit size of encoded data

        Print all the data for result analysis
}
```

2.2 Shannon-Fano Algorithm

Following pseudo-code represents the Shannon-Fano Code algorithm:

```
// Structure to store a node information
struct node
{
        char symbol
        int occurrence
        float frequency
        int array[]
        int top
}

// array of nodes
node p[]


//Procedure to build Shannon-Fano Tree:
buildShannonFanoTree (index left, index right, node p[])
{
        int total, halfTotal, leftTotal
        index k

        // Should not happen but checking if this for any error state
        if(left = right OR left > right)
                return
        // Last two nodes are reached
        if(left+1 = right)
                Increment array for left node with 0 and right node with 1
                Increment the top by 1 for both, left and right nodes
                return
        // else if more than two nodes are left
        else
                split complete list or smaller part of it into two equal halves
                k = index at split

        // Assign 0 to the left side nodes from k
        for ( i = l to k)
                p->array[i] = 0

        // Assign 1 to the right side nodes from k
```

```
        for ( i = k+1 to h)
                p->array[i] = 1

        // Recursive calls to buildShannonFanoTree function
        buildShannonFanoTree(l, k, p);
        buildShannonFanoTree(k + 1, h, p);
}
```

// Procedure to sort the symbols based on their frequency

```
sortByFrequency(int n, node p[])
{
        node temp
        for(i = 1 to n)
                for(j = 1 to n)
                        if(frequency at jth node < frequency at j+1th node)
                                data of temp node = data of jth node
                                data of jth node = data of j+1th node
                                data of j+1th node = data of temp node
}
```

// Main() Procedure

```
main(argc, argv)
{
        char sym[]
        float freq[]
        int occurance[]

        // File text to encode
        filename = read from the command line argument

        // read each character and its occurrence from the file
        while(till character in filename)
        {
                ith sym = character
                ith occurance = occurrence of character
        }

        n = no. of characters

        // calculate total occurrence
        for(i = 1 to n)
                total occurrence =  total occurrence + occurrence of ith character

        // calculate frequency of each character
        for(i = 1 to n)
                ith freq = occurrence of ith character / total occurrence

        // calculate entropy of each character
        for(i = 1 to n)
                entropy = entropy + (-ith freq * log2 ith freq)

        // calculate minimum bit length required to store each symbol based on fixed length coding
        for(i = 1 to n)
        {
                if(2^i >= n)
```

```
                fixed length code = i
        }

        // Calling sorting procedure
        sortByFrequency(n, p)

        // Call Shannon-Fano code procedure
        buildShannonFanoTree(0, n, p)

        // Calculate following statistics
        Total bit size of original data before encoding = total occurrences * 8 bits/ASCII character
        Total bit size of encoded data = total occurrences each character * encoded bit size of that character
        Efficiency of fixed length code = entropy / fixed length code
        Efficiency of encoded characters = entropy / average bit size of encoded characters
        Compression ratio  = Total bit size of original data before encoding / Total bit size of encoded data

        Print all the data for result analysis
}
```

3. Analysis of Used Algorithms

Following analysis matrix shows the characteristics and differences between Huffman and Shannon-Fano methods.

Criteria	Huffman Coding	Shannon-Fano Coding
Main Concept	It is purely based on the **frequency of occurrence of a character** used in the data file and does not care of the distribution of the data	It is based on the **Cumulative Distribution Function**
Algorithm Basics	1. Huffman coding uses Binary Tree. 2. It is **Bottom-Up approach** 3. It uses **priority-queue** with minimum heap i.e., heap with lowest frequency symbols with highest priority 4. At each iterative step, it extracts two nodes with minimum frequency from the priority queue and place node with lower priority to left and higher to the right. It then creates a new node with sum of those two node frequencies and adds that to the priority queue 5. Assign the value 0 to the left nodes and 1 to the right nodes	1. Shannon-Fano coding uses Binary Tree. 2. It is **Top-Down approach** 3. It uses a **sorted list** of symbols with their frequencies in decreasing order 4. At each recursive step, it splits the list into two parts, with the total probability of both the parts being as close to each other possible 5. Assign the value 0 to the left nodes and 1 to the right nodes
Distribution of Data	Distribution of frequency does not affect the performance of this algorithm	This algorithm requires balanced cumulative distribution of frequency for best performance
Optimality	Since it solely uses the frequency of occurrence and has no dependency on distribution, it **guarantees optimal** code	Since it is based on the cumulative distribution of the frequency, it **does not guarantee optimal** code
Efficiency & Compression Ratio	It **always provides good Efficiency** and **Compression Ratio** as the code is optimal	It **does not ALWAYS provide good Efficiency** and **Compression Ratio** as optimality is not guaranteed
Time Complexity	Given n nodes (i.e., n characters) Heapifying n nodes requires O(n) time. In every iteration we take out 2 nodes and insert 1 node. Therefore, for n nodes, it will take n-1 iterations to make the tree Extract min from heap twice O(log n)	Given n nodes (i.e., n characters) The recurrence relation is the following : $T(n)=T(k)+T(n-k)+\Theta(n)$ where k and n-k are the size of the partitions, k > 0

	Insert x in heap O(log n) Hence, the Every time complexity is **Θ(n*log n)**	If the partitions can be divided in such a way that the sizes are almost equal, for example if the probabilities are {0.2,0.25,0.25,0.3}, then the recurrence relation becomes: T(n)=2T(n/2)+Θ(n) Here, the Best Case time complexity turns out to be **Θ(n*log n)** The partitions done each time can be very unbalanced, for example if the probabilities are {0.05,0.1,0.15,0.2, 0.5}, then the recurrence relation becomes: T(n)=T(n-1)+T(1)+Θ(n) So, Worst Case time complexity turns out to be **Θ(n^2)**

4. Result Prediction and Preliminary Recommendation

As we noted in the section 3 above, the Shannon-Fano algorithm relies on Cumulative Distribution Function, it doesn't guarantee the best possible efficiency and compression ratio. On the other hand, Huffman algorithm solely relies on frequency of the samples and its performance is completely independent of the distribution of the data. Looking at this fact, the Huffman algorithm should always produce the optimal compression and the efficiency should be same or higher than that of the Shannon-Fano algorithm but never less than that.

Moreover, the time complexity of Shannon-Fano algorithm is always same or higher than Huffman algorithm, so expected time taken to run the algorithm program for Huffman algorithm should be same or less than that of Shannon-Fano.

Looking at the analysis, my prediction is that the test result will prove Huffman algorithm more efficient and optimal than Shannon-Fano algorithm for encoding the data and file compression. Hence, the recommendation is to use Huffman Algorithm due to its higher coding efficiency and less running time.

5. Performance Metrics for Algorithm Evaluation

Comparison of the Huffman and Shannon-Fano algorithms have been conducted using following performance criteria:

1) <u>Efficiency of the algorithm (η):</u>

   To understand the efficiency of the algorithm, we need to understand the Entropy. The entropy of the source of information is defined as following:

   $H = -\sum_{i=0}^{n} Freq[i] * \log_2 Freq[i]$; n is the number of characters

   Entropy tells us how many minimum bits are required to store/transfer the entire data file.

   Now, to compute the encoding algorithm efficiency (in %), we need to find the average length of algorithm code (*Lavg*) and divide the entropy (H) with that:

   $$\eta = \frac{H}{Lavg} * 100$$

2) <u>Compression Ratio:</u>

Compression Ratio of encoding algorithm is defined as

$$Compression\ Ratio\ (\%) = \frac{Total\ Compressed\ Data\ Bits}{Total\ Uncompressed\ Data\ Bits} * 100$$

Since the Huffman algorithm provides optimal code, we expect to see higher (or at least same but never less) the efficiency and compression ratio compared to that of Shannon-Fano.

3) Time complexity:

As we have seen in the section 3, the time complexity of Shannon-Fano algorithm will be somewhat same or in worst case higher than the time complexity of Huffman algorithm. Therefore, we can expect to see higher run time in executing Shannon-Fano algorithm code than that of Huffman.

6. Design Consideration

Choice of Programming Language:

Following are the reasons, C++ has been used as a programming language to develop code for the algorithms in this project:

- Easy to debug the code with freeware IDE e.g., CodeLite
- Freeware MingW compiler and linker
- Has sufficient library support to develop Huffman and Shannon-Fano algorithms
- Execution speed of C++ compiled programs are faster than many other languages
- Memory usage of compiled C++ program is quite optimized

Choice of Data Structures:

Arrays and C++ structures are used in both, Huffman and Shannon-Fano algorithms as they are easy and efficient to pass to the functions as parameters. For Huffman algorithm, the C++ *priority_queue* has been used which is helping to create a heap with max priority and is easy to compare, remove and add new data to the queue

7. Test Plan

7.1 Purpose

This section provides a test plan for validation of the C++ programs developed for Huffman and Shannon-Fano algorithms and to collect their performance metrices.

7.2 Testing Types

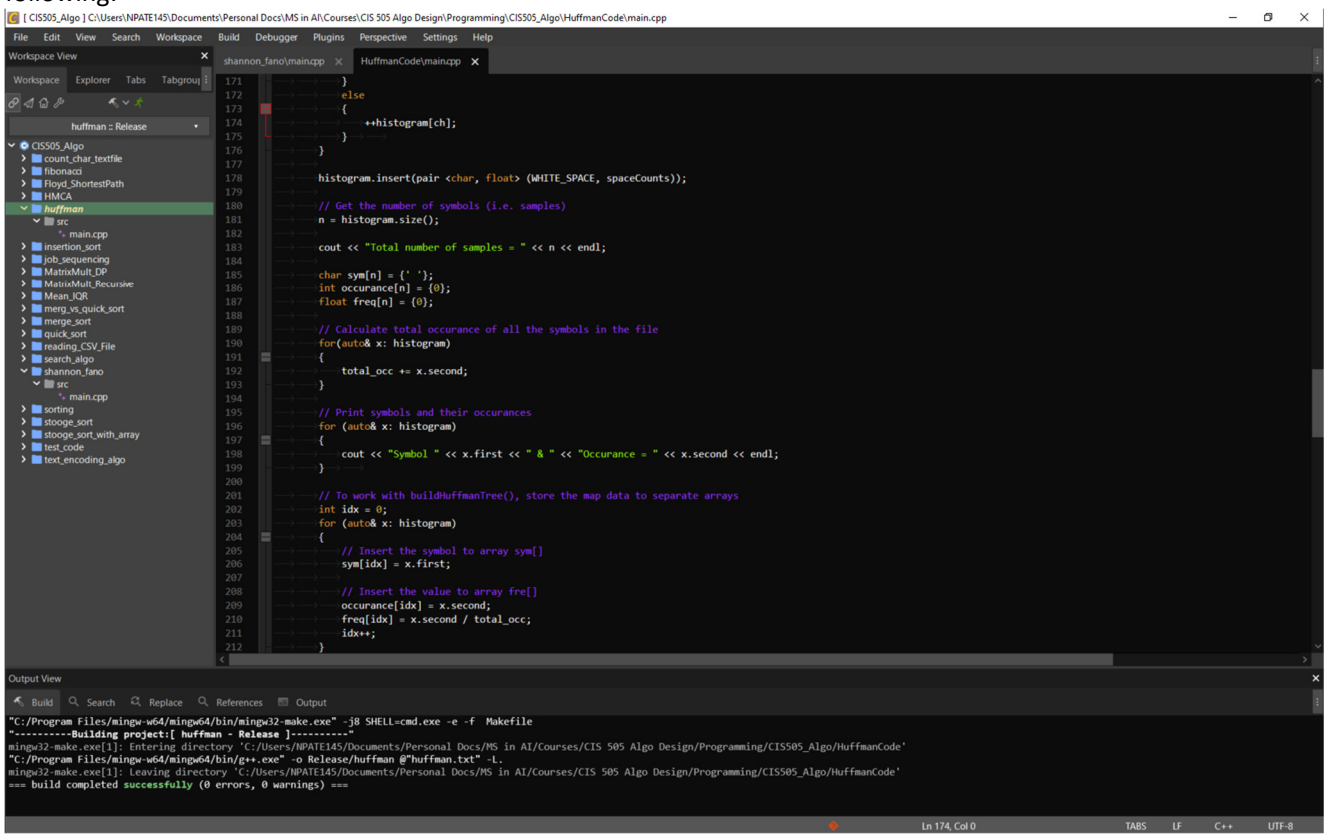
Following different types of testing that needs to be done to validate the programs and get the desired performance parameters:

7.2.1 Unit Testing

Unit testing is performed to verify if the desired result is achieved out of the programs. A small database is used to verify the working code and find any errors/warnings.

Following steps are followed:

1. Add test code with a matrix with only a limited data. For example, 5 to 6 characters and their frequency of occurrence
2. Build each project (Huffman and shannon_fano) separately and generate the executable file. For example, see following:



3. Check for any error and/or warning and if there is any, please fix the code and re-build until no error or warning is left.
4. Run the output executable file either using the CodeLite IDE option or in the Command Shell.
5. Check the printed output and compare with manually calculated codes for each character. If not getting desired result, please debug the code and fix any issue and then repeat steps 2 to 4 again until a desired result is achieved
6. Repeat step 5 for a few more times to confirm a repetitive behavior.

7.2.2 Functional Testing

Functional testing is performed to validate actual algorithm performance using data files with variable size as input.

Following steps are followed:

1. Remove the test code added during unit testing
2. Add the code to check for command line arguments. The idea is to pass each sample text file as part of command line argument and use it at a time.
3. Build each project (Huffman and Shannon_fano) separately and generate the executable file. For example, see following:

Name	Date modified	Type	Size
huffman	12/3/2021 3:40 PM	Application	259 KB
.d	12/3/2021 3:40 PM	D File	1 KB
main.cpp.o.d	12/3/2021 3:40 PM	D File	1 KB
main.cpp.o	12/3/2021 3:40 PM	O File	332 KB
Beggars_Opera	11/22/2021 4:47 PM	Text Document	116 KB
five_organge_pie	11/22/2021 2:30 PM	Text Document	41 KB
large_text_file_2_compress	11/22/2021 5:12 PM	Text Document	6,276 KB
sly_fox	11/18/2021 11:10 AM	Text Document	9 KB

Name	Date modified	Type	Size
shannon_fano	11/27/2021 10:04 PM	Application	146 KB
.d	11/27/2021 10:04 PM	D File	1 KB
main.cpp.o.d	11/27/2021 10:04 PM	D File	1 KB
main.cpp.o	11/27/2021 10:04 PM	O File	136 KB
Beggars_Opera	11/22/2021 4:47 PM	Text Document	116 KB
five_organge_pie	11/22/2021 2:30 PM	Text Document	41 KB
large_text_file_2_compress	11/22/2021 5:12 PM	Text Document	6,276 KB
sly_fox	11/18/2021 11:10 AM	Text Document	9 KB

4. Check for any error and/or warning and if there is any, please fix the code and re-build until no error or warning is left.
5. Run the output executable file in the Command Shell and pass each text file to encode one at a time.

```
PS C:\Users\NPATE145\Documents\Personal Docs\MS in AI\Courses\CIS 505 Algo Design\Programming\CIS505_Algo\HuffmanCode\Release
```

   The list of those files are as follows:

Beggars_Opera	11/22/2021 4:47 PM	Text Document	116 KB
five_organge_pie	11/22/2021 2:30 PM	Text Document	41 KB
large_text_file_2_compress	11/22/2021 5:12 PM	Text Document	6,276 KB
sly_fox	11/18/2021 11:10 AM	Text Document	9 KB

6. Note down the following performance parameters from the printed output on Command Shell for each of the above text files:

   Total original bit length
   Total encoded bit length
   Entropy
   Binary code length based on min fixed length coding
   Average Bit length after encoding
   Efficiency for binary code
   Efficiency of coding algo
   Compression ratio for file xyx.txt
   Total Execution Time for Huffman encoding algo (in microseconds) for xyx.txt

8. Source Code of Programs

<u>Source code for Huffman Algorithm:</u>

```
/******* AUTHOR: Nayan Patel *****************
******* COURSE: CIS 505-002 ******************
******* TERM: Fall 2021 **********************
******* Faculty: Prof. Thomas Steiner ********
******* Project: Item 1 **********************
******* Program Name: C++ Code for ***********
******* Huffman Encoding Algo ***************/

// Including header files
#include <bits/stdc++.h>
#include <chrono>

using namespace std;
using namespace chrono;

#define CHAR_BIT_LENGTH 8
#define WHITE_SPACE     32

map<char, string> encodedBits;

/* Structure for Huffman tree node */
typedef struct node
{
    // symbol
    char sym;

    // Frequency of the symbol
    float freq;

    // Left and right child node

    node *left, *right;

    node(char sym, float freq)
    {
        left = right = NULL;
        this->sym = sym;
        this->freq = freq;
    }
} node;

/* Comparing two heap nodes */
struct compare
{
    bool operator()(node* l, node* r)
    {
        return (l->freq > r->freq);
    }
};

/* Get the code for each symbol from the priority queue */
void getCodes(struct node* root, string str)
{
    if (!root)
        return;

    if (root->sym != '$')
    {
        encodedBits.insert(pair <char, string> (root->sym, str));
    }

    getCodes(root->left, str + "0");
```

```
    getCodes(root->right, str + "1");
}

/* Function that builds a Huffman Tree */
void buildHuffmanTree(char sym[], float freq[], int size)
{
    struct node *left, *right, *top;

    // Create a priority queue & inserts all characters of sym[]
    priority_queue <node*, vector<node*>, compare> PQ;

    for (int i = 0; i < size; ++i)
    {
        PQ.push(new node(sym[i], freq[i]));
    }

    // Iterate while size of priority queue doesn't become 1
    while (PQ.size() != 1)
    {

        // Extract the two minimum
        // frequency items from priority queue
        left = PQ.top();
        PQ.pop();

        right = PQ.top();
        PQ.pop();

        // Create a new internal node with
        // frequency equal to the sum of the
        // two nodes frequencies. Make the
        // two extracted node as left and right children
        // of this new node. Add this node
        // to the priority queue '$' is a special value
        // for internal nodes, not used
        top = new node('$', left->freq + right->freq);

        top->left = left;
        top->right = right;

        PQ.push(top);
    }

    // Get Huffman codes using
    // the Huffman tree built above
    getCodes(PQ.top(), "");
}

/* Function to sort the symbols based on their probability or frequency
 * NOTE:  This is purely for printing the results and NOT USED in the
 * actual Huffman algorithm */
void sortByFrequency(char sym[], float freq[], int n)
{
    char tempData = ' ';
    float tempFreq = 0.0;
    for (int j = 0; j < n; j++)
    {
        for (int i = 0; i < n - 1; i++)
        {
            if (freq[i] < freq[i + 1])
            {
                tempFreq = freq[i];
```

```
                tempData = sym[i];

                freq[i] = freq[i + 1];
                sym[i] = sym[i + 1];

                freq[i + 1] = tempFreq;
                sym[i + 1] = tempData;
            }
        }
    }
}

/* Main function to drive the Huffman code Contains two arguments: (1) the executable,
* (2) input file name that needs compression */
int main(int argc, char* argv[])
{
    cout << "Number of argeuments is = " << argc << " and input file for compression is
"<< argv[1] << endl;
    string fileName = argv[1];

    /* Input arguments have to be exact 2 */
    if(argc == 2)
    {
        char ch = 0;
        int n = 0;
        int total_occ = 0;
        int spaceCounts = 0;
        long originalTotalBits = 0;
        long encodedTotalBits = 0;
        float entropy = 0.0;
        float fixedLengthCode = 0.0;
        float avgBits = 0.0;
        float efficAfterCompress = 0.0;
        float efficBeforeCompress = 0.0;
        float compression = 0.0;

        ifstream in("C:/Users/NPATE145/Documents/Personal Docs/MS in AI/Courses/CIS 505
Algo Design/Project/Sample Database/" + fileName);

        // Map to store counts of each symbol.
        map<char, float> histogram;

        // Read each symbol from the input file and store it in the map
        while (in.get(ch))
        {
            if (isspace(ch))
            {
                ++spaceCounts;
            }
            else
            {
                ++histogram[ch];
            }
        }

        histogram.insert(pair <char, float> (WHITE_SPACE, spaceCounts));

        // Get the number of symbols (i.e. samples)
        n = histogram.size();

        cout << "Total number of samples = " << n << endl;
```

```
        char sym[n] = {' '};
        int occurance[n] = {0};
        float freq[n] = {0};

        // Calculate total occurance of all the symbols in the file
        for(auto& x: histogram)
        {
            total_occ += x.second;
        }

        // Print symbols and their occurances
        for (auto& x: histogram)
        {
            cout << "Symbol " << x.first << " & " << "Occurrence = " << x.second <<
endl;
        }

        // To work with buildHuffmanTree(), store the map data to separate arrays
        int idx = 0;
        for (auto& x: histogram)
        {
            // Insert the symbol to array sym[]
            sym[idx] = x.first;

            // Insert the value to array fre[]
            occurance[idx] = x.second;
            freq[idx] = x.second / total_occ;
            idx++;
        }

        // Calculating entropy of symbols based on their frequency
        for(int i = 0; i < n; i++)
        {
            entropy += (-( freq[i] * log2 (freq[i]) ) );
        }

        // Minimum bit length required to store each symbol based on fixed length coding
        for(int i = 0; i < n; i++)
        {
            if(pow(2, i) >= n)
            {
                fixedLengthCode = i;
                break;
            }
        }

        // Start the clock to measure the Huffman algorithm timing
        auto start = high_resolution_clock::now();

        // Call buildHuffmanTree() function and pass symbol, frequency and number of
samples
        buildHuffmanTree(sym, freq, n);

        // Get number of encoded character bits for each symbol
        int encodedCharBits[n] = {0};
        for(int i = 0; i < n; i++)
        {
            encodedCharBits[i] = encodedBits[sym[i]].length();
        }

        // Calculate total bits required to store each character without
        // and with encoding
```

```
        for(int i = 0; i < n; i++)
        {
            originalTotalBits += occurance[i] * CHAR_BIT_LENGTH;
            encodedTotalBits += occurance[i] * encodedCharBits[i];
        }

        // Efficiency before the compression
        efficBeforeCompress = (entropy / fixedLengthCode) * 100.0;
        // Average bit size of encoded symbols
        avgBits = (float)encodedTotalBits / (float)total_occ;
        // Efficiency after the compression
        efficAfterCompress = (entropy / avgBits) * 100.0;
        // % Compression
        compression = ((float)originalTotalBits) / (float)encodedTotalBits * 100.0;

        /********** Print all the useful information calculated previously ***********/
        sortByFrequency(sym, freq, n);
        cout << "Symbol    " << "Frequency\t" << "code\t\t" << endl;
        for(int i = 0; i < n; i++)
        {
            cout << sym[i] << "\t" << freq[i] << "\t" << encodedBits[sym[i]] << endl;
        }

        cout << endl;
        cout << "Total original bit length is " << originalTotalBits << endl;
        cout << "Total encoded bit length is " << encodedTotalBits << endl;
        cout << "Entropy is " << entropy << endl;
        cout << "Binary code length based on min fixed length coding = " <<
fixedLengthCode << endl;
        cout << "Average Bit length after encoding is " << avgBits << endl;
        cout << "Efficiency for binary code is " << efficBeforeCompress << endl;
        cout << "Efficiency of Huffman coding is " << efficAfterCompress << endl;

        cout << "Compression ratio for file " << fileName << " is " << compression <<
endl;

        // Stop the clock and calculate the delta to measure the actual execution time
for Huffman coding
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop - start).count();

        cout << "Total Execution Time for Huffman encoding algo (in microseconds) for
file "
            << fileName << " is " << duration << setprecision(9) << endl;
    }
    // If number of arguments are incorrect, print error message
    else
    {
        cout << "Wrong number of arguments, please check and run program with right
arguments" << endl;
    }

    return 0;
}
```

Source code for Shannon-Fano Algorithm:

```
/****** AUTHOR: Nayan Patel ******************
******* COURSE: CIS 505-002 ******************
******* TERM: Fall 2021 **********************
```

```
******* Faculty: Prof. Thomas Steiner ********
******* Project: Item 1 **********************
******* Program Name: C++ Code for ***********
******* Shannon-Fano Encoding Algo **********/

// include header files
#include <bits/stdc++.h>
#include <chrono>

using namespace std;
using namespace chrono;

#define HALF            2.0f
#define CHAR_BIT_LENGTH 8
#define WHITE_SPACE     32

/* Structure for Shannon-Fano tree node */
typedef struct node
{

    // for storing symbol
    string sym;

    // for storing frequency
    int occ;
    float freq;
    int arr[16];
    int top;
}node;

// An instance of the node structure to store symbolds, frequency, code and length of
code
struct node p[128];

/* Function that builds a shannon-fano Tree */
void buildShannonFanoTree(int l, int h, node p[])
{
    int i = 0, k = 0;
    float total = 0, halfTotal = 0, leftTotal = 0;
    \
    // If no more samples left for processing, return
    if (l == h || l > h)
    {
        return;
    }

    // If last two samples left for processing, assignm them the code and return
    if ( (l + 1) == h )
    {
        p[l].arr[++(p[l].top)] = 0;
        p[h].arr[++(p[h].top)] = 1;
        return;
    }
    // Otherwise, divide the samples into two as equal as possible parts
    else
    {
        for(i = l; i < h; i++)
        {
            total += p[i].freq;
        }

        halfTotal = total / HALF;
```

```
        for(i = l; i < h; i++)
        {
            leftTotal += p[i].freq;

            if(leftTotal >= halfTotal)
            {
                k = i;
                break;
            }
        }

        // Assing add 0 to all the left nodes
        for (i = l; i <= k; i++)
        {
            p[i].arr[++(p[i].top)] = 0;
        }

        // Assing add 1 to all the left nodes
        for (i = k + 1; i <= h; i++)
        {
            p[i].arr[++(p[i].top)] = 1;
        }

        // Recursive calls to buildShannonFanoTree function
        buildShannonFanoTree(l, k, p);
        buildShannonFanoTree(k + 1, h, p);
    }
}

/* Function to sort the symbols based on their frequency */
void sortByFrequency(int n, node p[])
{
    int i, j;
    node temp;
    for (j = 0; j < n; j++)
    {
        for (i = 0; i < n - 1; i++)
        {
            if ((p[i].freq) < (p[i + 1].freq))
            {
                temp.freq = p[i].freq;
                temp.occ = p[i].occ;
                temp.sym = p[i].sym;

                p[i].freq = p[i + 1].freq;
                p[i].occ = p[i + 1].occ;
                p[i].sym = p[i + 1].sym;

                p[i + 1].freq = temp.freq;
                p[i + 1].occ = temp.occ;
                p[i + 1].sym = temp.sym;
            }
        }
    }
}

/* function to print shannon codes */
void printSFCodes(int n, node p[])
{
    int i, j;
    cout << "\n\tSymbol\t\tFrequency\tCode";
```

```
    for (i = 0; i < n; i++)
    {
        cout << "\n\t" << p[i].sym << "\t\t" << p[i].freq << "\t";
        for (j = 0; j <= p[i].top; j++)
            cout << p[i].arr[j];
    }
}

/* Main function to drive the shannon-fano code Contains two arguments: (1) the
executable,
* (2) input file name that needs compression */
int main(int argc, char* argv[])
{
    cout << "Number of argeuments is = " << argc << " and input file for compression is
"<< argv[1] << endl;
    string fileName = argv[1];

    /* Input arguments have to be exact 2 */
    if(argc == 2)
    {
        char ch = 0;
        int n = 0;
        int total_occ = 0;
        int spaceCounts = 0;
        float total = 0;
        long originalTotalBits = 0;
        long encodedTotalBits = 0;
        float entropy = 0.0;
        float fixedLengthCode = 0.0;
        float avgBits = 0.0;
        float efficAfterCompress = 0.0;
        float efficBeforeCompress = 0.0;
        float compression = 0.0;

        ifstream in("C:/Users/NPATE145/Documents/Personal Docs/MS in AI/Courses/CIS 505
Algo Design/Project/Sample Database/" + fileName);

        // Map to store counts of each symbol.
        map<char, float> histogram;

        // Read each symbol from the input file and store it in the map
        while (in.get(ch))
        {
            if (isspace(ch))
            {
                ++spaceCounts;
            }
            else
            {
                ++histogram[ch];
            }
        }

        histogram.insert(pair <char, float> (WHITE_SPACE, spaceCounts));

        // Get the number of symbols (i.e. samples)
        n = histogram.size();

        cout << "Total number of samples = " << n << endl;

        // Calculate total occurance of all the symbols in the file
        for(auto& x: histogram)
```

```
        {
            total_occ += x.second;
        }

        // Print symbols and their occurances
        for (auto& x: histogram)
        {
            cout << "Symbol " << x.first << " & " << "Occurrence = " << x.second <<
endl;
        }

        // To work with buildShannonFanoTree(), store the map data to node p[]
        int idx = 0;
        for (auto& x: histogram)
        {
            // Insert the symbol to node
            p[idx].sym = x.first;

            // Insert the value to node
            p[idx].occ = x.second;
            p[idx].freq = x.second / total_occ;
            total = total + p[idx].freq;
            idx++;
        }

        // Calculating entropy of symbols based on their frequency
        for(int i = 0; i < n; i++)
        {
            entropy += (-( p[i].freq * log2 (p[i].freq) ) );
        }

        // Minimum bit length required to store each symbol based on fixed length coding
        for(int i = 0; i < n; i++)
        {
            if(pow(2, i) >= n)
            {
                fixedLengthCode = i;
                break;
            }
        }

        // Start the clock to measure the Huffman algorithm timing
        auto start = high_resolution_clock::now();

        // Sorting the symbols based on their frequency in decreasing order
        sortByFrequency(n, p);

        // Initializing the symbol code length to -1
        for (int i = 0; i < n; i++)
        {
            p[i].top = -1;
        }

        // Call buildShannonFanoTree() function and pass first and end sample index and
node p[]
        buildShannonFanoTree(0, n, p);

        // Calculate total bits required to store each character without
        // and with encoding
        for(int i = 0; i < n; i++)
        {
            originalTotalBits += p[i].occ * CHAR_BIT_LENGTH;
```

```
            encodedTotalBits += p[i].occ * (p[i].top + 1);
        }

        // Efficiency before the compression
        efficBeforeCompress = (entropy / fixedLengthCode) * 100.0;
        // Average bit size of encoded symbols
        avgBits = (float)encodedTotalBits / (float)total_occ;
        // Efficiency after the compression
        efficAfterCompress = (entropy / avgBits) * 100.0;
        // % Compression
        compression = ((float)originalTotalBits) / (float)encodedTotalBits * 100.0;

        /********** Print all the useful information calculated previously ***********/
        printSFCodes(n, p);
        cout << endl;
        cout << "Total original bit length is " << originalTotalBits << endl;
        cout << "Total encoded bit length is " << encodedTotalBits << endl;
        cout << "Entropy is " << entropy << endl;
        cout << "Binary code length based on min fixed length coding = " <<
fixedLengthCode << endl;
        cout << "Average Bit length after encoding is " << avgBits << endl;
        cout << "Efficiency for binary code is " << efficBeforeCompress << endl;
        cout << "Efficiency of shannon-fano coding is " << efficAfterCompress << endl;

        cout << "Compression ratio for file " << fileName << " is " << compression <<
endl;

        // Stop the clock and calculate the delta to measure the actual execution time
for Huffman coding
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop - start).count();

        cout << "Total Execution Time for Shannon-Fano encoding algo (in microseconds)
for file " << fileName << " is " << duration << setprecision(9) << endl;
    }
    // If number of arguments are incorrect, print error message
    else
    {
        cout << "Wrong number of arguments, please check and run program with right
arguments" << endl;
    }

    return 0;
}
```

9. Build Results and Output of Programs

Following are the build logs of Huffman and Shannon-Fano algorithm's programs (main.cpp) files. Please note that the GCC based MingW compiler and linker were used to create this build.

Build Log and Sample Output for Huffman Algorithm C++ Code:

```
"C:/Program Files/mingw-w64/mingw64/bin/mingw32-make.exe" -j8 SHELL=cmd.exe -e -f  Makefile
"----------Building project:[ Huffman - Release ]----------"
mingw32-make.exe[1]: Entering directory 'C:/Users/NPATE145/Documents/Personal Docs/MS in AI/Courses/CIS 505 Algo
Design/Programming/CIS505_Algo/HuffmanCode'
```

```
"C:/Program Files/mingw-w64/mingw64/bin/g++.exe"  -c  "C:/Users/NPATE145/Documents/Personal Docs/MS in AI/Courses/CIS 505 Algo Design/Programming/CIS505_Algo/HuffmanCode/main.cpp" -Wall -DNDEBUG  -o Release/main.cpp.o -I. -I.
"C:/Program Files/mingw-w64/mingw64/bin/g++.exe" -o Release/Huffman @"huffman.txt" -L.
mingw32-make.exe[1]: Leaving directory 'C:/Users/NPATE145/Documents/Personal Docs/MS in AI/Courses/CIS 505 Algo Design/Programming/CIS505_Algo/HuffmanCode'
=== build completed successfully (0 errors, 0 warnings) ===
```

```
PS C:\Users\NPATE145\Documents\Personal Docs\MS in AI\Courses\CIS 505 Algo Design\Programming\CIS505_Algo\HuffmanCode\Release> ./Huffman sly_fox.txt

Number of arguments is = 2 and input file for compression is sly_fox.txt

Total number of samples = 55

Symbol   & Occurrence = 1897
Symbol ! & Occurrence = 19
Symbol " & Occurrence = 74
Symbol ' & Occurrence = 6
Symbol , & Occurrence = 99
Symbol - & Occurrence = 3
Symbol . & Occurrence = 109
Symbol : & Occurrence = 3
Symbol ? & Occurrence = 8
Symbol A & Occurrence = 7
Symbol B & Occurrence = 3
Symbol C & Occurrence = 3
Symbol D & Occurrence = 3
Symbol E & Occurrence = 4
Symbol F & Occurrence = 30
Symbol G & Occurrence = 7
Symbol H & Occurrence = 17
Symbol I & Occurrence = 14
Symbol L & Occurrence = 4
Symbol M & Occurrence = 38
Symbol N & Occurrence = 5
Symbol O & Occurrence = 8
Symbol P & Occurrence = 10
Symbol R & Occurrence = 35
Symbol S & Occurrence = 39
Symbol T & Occurrence = 13
Symbol U & Occurrence = 1
Symbol W & Occurrence = 16
Symbol Y & Occurrence = 1
Symbol a & Occurrence = 447
Symbol b & Occurrence = 147
Symbol c & Occurrence = 112
Symbol d & Occurrence = 301
Symbol e & Occurrence = 694
Symbol f & Occurrence = 121
Symbol g & Occurrence = 179
Symbol h & Occurrence = 403
Symbol i & Occurrence = 385
Symbol j & Occurrence = 7
Symbol k & Occurrence = 55
Symbol l & Occurrence = 273
```

Symbol m & Occurrence = 114
Symbol n & Occurrence = 352
Symbol o & Occurrence = 478
Symbol p & Occurrence = 99
Symbol q & Occurrence = 13
Symbol r & Occurrence = 352
Symbol s & Occurrence = 304
Symbol t & Occurrence = 495
Symbol u & Occurrence = 135
Symbol v & Occurrence = 41
Symbol w & Occurrence = 126
Symbol x & Occurrence = 35
Symbol y & Occurrence = 140
Symbol z & Occurrence = 2

Symbol	Frequency	code
	0.22894	01
e	0.0837557	1110
t	0.0597393	1001
o	0.0576877	1000
a	0.0539464	0011
h	0.0486363	0001
i	0.0464639	0000
n	0.0424813	11011
r	0.0424813	11110
s	0.0366884	11010
d	0.0363263	11000
l	0.0329471	10110
g	0.0216027	111110
b	0.0177408	110010
y	0.016896	101110
u	0.0162925	101011
w	0.0152064	101010
f	0.0146029	101001
m	0.0137581	001011
c	0.0135168	001010
.	0.0131547	001001
,	0.0119479	1111110
p	0.0119479	1111111
"	0.00893073	1011111
k	0.0066377	0010001
v	0.00494811	11001111
S	0.00470673	11001110
M	0.00458605	11001101
R	0.00422399	10111101
x	0.00422399	10111100
F	0.00362056	10100001
!	0.00229302	110011001
H	0.00205165	101000111
W	0.00193097	101000101
I	0.0016896	101000100
T	0.00156891	001000010
q	0.00156891	001000011
P	0.00120685	1100110001
?	0.000965484	1100110000

```
O      0.000965484     1010001101
A      0.000844798     1010000001
G      0.000844798     1010000010
j      0.000844798     1010000011
'      0.000724113     0010000001
N      0.000603427     0010000000
E      0.000482742     10100000001
L      0.000482742     10100011001
-      0.000362056     00100000100
:      0.000362056     00100000111
B      0.000362056     10100000000
C      0.000362056     00100000101
D      0.000362056     00100000110
z      0.000241371     101000110000
U      0.000120685     1010001100010
Y      0.000120685     1010001100011

Total original bit length is 66288
Total encoded bit length is 36154
Entropy is 4.33737
Binary code length based on min fixed length coding = 6
Average Bit length after encoding is 4.36326
Efficiency for binary code is 72.2894
Efficiency of Huffman coding is 99.4065
Compression ratio for file sly_fox.txt is 183.349
Total Execution Time for Huffman encoding algo (in microseconds) for file sly_fox.txt is 21897
```

Build Log and Sample Output for Shannon-Fano Algorithm C++ Code:

```
"C:/Program Files/mingw-w64/mingw64/bin/mingw32-make.exe" -j8 SHELL=cmd.exe -e -f  Makefile
"----------Building project:[ shannon_fano - Release ]----------"
mingw32-make.exe[1]: Entering directory 'C:/Users/NPATE145/Documents/Personal Docs/MS in AI/Courses/CIS 505 Algo
Design/Programming/CIS505_Algo/shannon_fano'
"C:/Program Files/mingw-w64/mingw64/bin/g++.exe"  -c  "C:/Users/NPATE145/Documents/Personal Docs/MS in
AI/Courses/CIS 505 Algo Design/Programming/CIS505_Algo/shannon_fano/main.cpp" -Wall -DNDEBUG  -o
Release/main.cpp.o -I. -I.
"C:/Program Files/mingw-w64/mingw64/bin/g++.exe" -o Release/shannon_fano @"shannon_fano.txt" -L.
mingw32-make.exe[1]: Leaving directory 'C:/Users/NPATE145/Documents/Personal Docs/MS in AI/Courses/CIS 505 Algo
Design/Programming/CIS505_Algo/shannon_fano'
=== build completed successfully (0 errors, 0 warnings) ===
```

```
PS C:\Users\NPATE145\Documents\Personal Docs\MS in AI\Courses\CIS 505 Algo
Design\Programming\CIS505_Algo\shannon_fano\Release> ./shannon_fano sly_fox.txt

Number of argeuments is = 2 and input file for compression is sly_fox.txt

Total number of samples = 55

Symbol   & Occurrence = 1897
Symbol ! & Occurrence = 19
Symbol " & Occurrence = 74
Symbol ' & Occurrence = 6
Symbol , & Occurrence = 99
Symbol - & Occurrence = 3
Symbol . & Occurrence = 109
Symbol : & Occurrence = 3
```

Symbol ? & Occurrence = 8
Symbol A & Occurrence = 7
Symbol B & Occurrence = 3
Symbol C & Occurrence = 3
Symbol D & Occurrence = 3
Symbol E & Occurrence = 4
Symbol F & Occurrence = 30
Symbol G & Occurrence = 7
Symbol H & Occurrence = 17
Symbol I & Occurrence = 14
Symbol L & Occurrence = 4
Symbol M & Occurrence = 38
Symbol N & Occurrence = 5
Symbol O & Occurrence = 8
Symbol P & Occurrence = 10
Symbol R & Occurrence = 35
Symbol S & Occurrence = 39
Symbol T & Occurrence = 13
Symbol U & Occurrence = 1
Symbol W & Occurrence = 16
Symbol Y & Occurrence = 1
Symbol a & Occurrence = 447
Symbol b & Occurrence = 147
Symbol c & Occurrence = 112
Symbol d & Occurrence = 301
Symbol e & Occurrence = 694
Symbol f & Occurrence = 121
Symbol g & Occurrence = 179
Symbol h & Occurrence = 403
Symbol i & Occurrence = 385
Symbol j & Occurrence = 7
Symbol k & Occurrence = 55
Symbol l & Occurrence = 273
Symbol m & Occurrence = 114
Symbol n & Occurrence = 352
Symbol o & Occurrence = 478
Symbol p & Occurrence = 99
Symbol q & Occurrence = 13
Symbol r & Occurrence = 352
Symbol s & Occurrence = 304
Symbol t & Occurrence = 495
Symbol u & Occurrence = 135
Symbol v & Occurrence = 41
Symbol w & Occurrence = 126
Symbol x & Occurrence = 35
Symbol y & Occurrence = 140
Symbol z & Occurrence = 2

Symbol	Frequency	Code
	0.22894	000
e	0.0837557	001
t	0.0597393	0100
o	0.0576877	0101
a	0.0539464	0110
h	0.0486363	0111

i	0.0464639	1000
n	0.0424813	10010
r	0.0424813	10011
s	0.0366884	1010
d	0.0363263	10110
l	0.0329471	10111
g	0.0216027	11000
b	0.0177408	110010
y	0.016896	110011
u	0.0162925	110100
w	0.0152064	110101
f	0.0146029	110110
m	0.0137581	110111
c	0.0135168	111000
.	0.0131547	111001
,	0.0119479	111010
p	0.0119479	1110110
"	0.00893073	1110111
k	0.0066377	1111000
v	0.00494811	11110010
S	0.00470673	11110011
M	0.00458605	1111010
R	0.00422399	11110110
x	0.00422399	11110111
F	0.00362056	11111000
!	0.00229302	11111001
H	0.00205165	111110100
W	0.00193097	111110101
I	0.0016896	111110110
T	0.00156891	111110111
q	0.00156891	111111000
P	0.00120685	111111001
?	0.000965484	1111110100
O	0.000965484	1111110101
A	0.000844798	1111110110
G	0.000844798	1111110111
j	0.000844798	1111111000
'	0.000724113	1111111001
N	0.000603427	1111111010
E	0.000482742	11111110110
L	0.000482742	11111110111
-	0.000362056	11111111000
:	0.000362056	11111111001
B	0.000362056	11111111010
C	0.000362056	11111111011
D	0.000362056	11111111100
z	0.000241371	11111111101
U	0.000120685	11111111110
Y	0.000120685	111111111110

Total original bit length is 66288
Total encoded bit length is 36703
Entropy is 4.33737
Binary code length based on min fixed length coding = 6
Average Bit length after encoding is 4.42952

Efficiency for binary code is 72.2894
Efficiency of shannon-fano coding is 97.9196
Compression ratio for file sly_fox.txt is 180.606
Total Execution Time for Shannon-Fano encoding algo (in microseconds) for file sly_fox.txt is 58864

Summary of Results:

Original Data						Performance Parameters (from Encoded Data)									
File Name	Total No of Characters	Entropy (H)	Total Original Bit Length	Fixed Length Code for original data	Efficiency of Fixed Length Coding (%)	Total Encoded Bit Length		Average Bit Length After Encoding		Efficiency of code (η)		Compression ratio (%)		Time to Execute Program (in µSec)	
						Huffman	Shannon-Fano	Huffman	Shannon-Fano	Huffman	Shannon-Fano	Huffman	Shannon-Fano	Huffman	Shannon-Fano
sly_fox.txt	55	4.33737	66288	6	72.2894	36154	36703	4.36326	4.42952	99.4065	97.9196	183.349	180.606	21897	58864
five_organge_pie.txt	74	4.368	322048	7	62.4	177520	178321	4.40978	4.42968	99.0526	98.6077	181.415	180.6	28923	67078
Beggars_Opera.txt	81	4.3146	928472	7	61.6371	504704	505222	4.34868	4.35315	99.2161	99.1144	183.964	183.775	32912	72896
large_text_file_2_compress.txt	82	5.26428	51048608	7	75.204	33759779	33807834	5.29061	5.29814	99.5023	99.3609	151.211	150.996	33908	80750

10. Result Analysis

Following values are derived when calculated the relative difference in the performance parameters of both Huffman and Shannon-Fano algorithms:

Efficiency of code (η)			Compression ratio (%)			Time to Execute Program (in µSec)		
Huffman	Shannon-Fano	% Relative Difference $\frac{(H-SF)}{SF} * 100$	Huffman	Shannon-Fano	% Relative Difference $\frac{(H-SF)}{SF} * 100$	Huffman	Shannon-Fano	% Relative Difference $\frac{(SF-H)}{H} * 100$
99.4065	97.9196	1.5185	183.349	180.606	1.5188	21897	58864	168.82
99.0526	98.6077	0.4512	181.415	180.6	0.4513	28923	67078	131.92
99.2161	99.1144	0.1026	183.964	183.775	0.1028	32912	72896	121.49
99.5023	99.3609	0.1423	151.211	150.996	0.1424	33908	80750	138.14

Major observations are made from the above results:

- The Huffman algorithm always outputs higher efficiency and compression ratio compared to that of Shannon-Fano algorithms. That is expected (see section 4) as the Huffman algorithm guarantees optimal result whereas Shannon-Fano does not
- The efficiency and compression ratio values do not vary with the size of the input data
- The speed of the Shannon-Fano algorithm is significantly higher than that of Huffman algorithm. It also meets our original prediction (section 4).

The results of both the algorithms meet with our original prediction, and our recommendation turns out to be correct i.e. the recommendation is to use Huffman Algorithm due to its higher coding efficiency and less running time.

11. Conclusion

An experimental comparison of Huffman and Shannon-Fano algorithm for encoding the data from text files is carried out. By considering the efficiency of the algorithm, compression ratio and the speed of both algorithms as criteria for this comparison, the Huffman algorithm has proved out to be the most efficient. The most attractive aspect of Huffman algorithm which was learned here is its guaranteed optimality, so even though no big difference is observed, it is non-trivial as even a small amount of higher compression ratio means higher saving of storage space and lesser the cost.